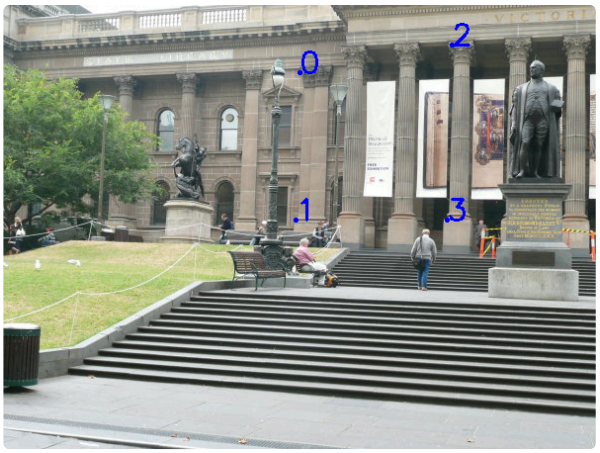
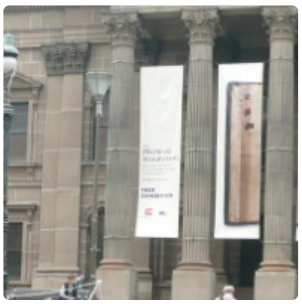
Scipy

```
|---------->x
|Opencv
|
|
v y
```

1. scipy.interpolate.RegularGridInterpolator

```
if __name__ == '__main__':
    # Read image
    img = cv2.imread('library1.jpg')
    # Display image
    cv2.imshow('img', img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

    """

    |---------->y       |---------->x
    | numpy             |Opencv
    |                   |
    |                   |
    v x                 v y

    """
    # img.shape: (H, W)
    x = np.linspace(0, img.shape[0]-1, img.shape[0])
    y = np.linspace(0, img.shape[1]-1, img.shape[1])

    interp = RegularGridInterpolator((x, y), img, bounds_error=False, fill_value=None)

    # pts are collected using the OpenCV function cv2.setMouseCallback('image', click_event)
    # which returns (x, y) in OpenCV coordinates.
    pts = np.array([[300, 68], [296, 217], [454, 40], [449, 217]])  # shape (n, d),
                                                                    # (x, y) in OpenCV coordinates
    color = (255, 0, 0)
    for i in range(pts.shape[0]):
        # Tip: everytime we're using OpenCV functions we should conform to OpenCV coordinates.
        cv2.circle(img, (int(pts[i, 0]), int(pts[i, 1])), 1, color, 2)
        cv2.putText(img, str(i), (int(pts[i, 0]), int(pts[i, 1])), cv2.FONT_HERSHEY_SIMPLEX, 1, color, 2)
    # Display points
    cv2.imshow('img', img)
    cv2.waitKey(0)
    cv2.imwrite('library_with_points.png', img)
    cv2.destroyAllWindows()

    half_win = 100
    print(pts.shape)
    center_y, center_x = np.mean(pts, axis=0)
    print("centers", center_x, center_y)

    xx = np.linspace(center_x-half_win, center_x+half_win, 2*half_win-1)
    yy = np.linspace(center_y-half_win, center_y+half_win, 2*half_win-1)

    # xx = np.linspace(pts[0, 1], pts[3, 1], 2*half_win-1)
    # yy = np.linspace(pts[0, 0], pts[3, 0], 2*half_win-1)

    X, Y = np.meshgrid(xx, yy, indexing='ij')
    w1 = interp((X, Y))

    w1=w1.astype(np.uint8)
    print(w1.shape)
    print(np.max(w1))
    cv2.imshow('w1', w1)
    cv2.waitKey(0)
    cv2.imwrite('window_w1.png', w1)
    cv2.destroyAllWindows()
```





2. scipy.spatial.transform.Rotation

```
def x2R(self, x):
    trans = [x[0], x[1], x[2], 1.0]
    R = np.eye(4)
    R[:3, :3] = Rotation.from_euler('xyz', [x[3], x[4], x[5]], degrees=False).as_matrix()
    R[ :, 3] = trans
    return R

def R2x(self, R):
    trans = R[:3, 3]
    rot = Rotation.from_matrix(R[:3, :3]).as_euler('xyz', degrees=False)
    x = np.hstack((trans, rot))
    return x

def inv_x(self, x):
    R = self.x2R(x)
    R_inv = np.linalg.inv(R)
    inv_x = self.R2x(R_inv)
    return inv_x
```